

1. Artificial Neural Networks (ANN)

1.1 Introduction to ANN

- **Definition:** An Artificial Neural Network (ANN) is a computational model designed to mimic the way biological neurons work in the human brain. It consists of layers of interconnected processing units (neurons) that learn patterns in data through a process of weight adjustment. ANNs are widely used in tasks like image recognition, language processing, and data classification.
- **Key Concept:** ANNs are composed of simple, interconnected units called neurons or **perceptrons** (the building blocks of ANNs). These neurons work together to transform input data into useful output, making the network capable of learning from experience.

1.2 Biological Inspiration

The concept of artificial neural networks comes from biological neurons found in animal brains So they share a lot of similarities in structure and function wise.

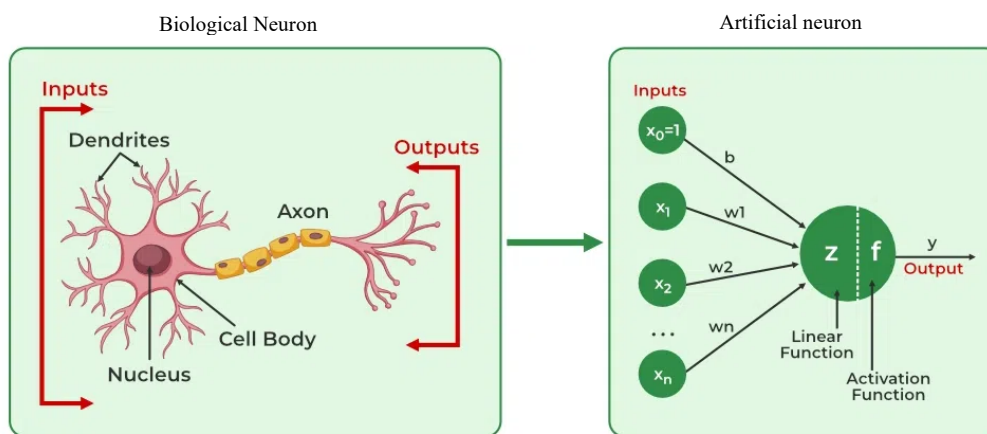


Figure 1 Biological neuron to Artificial neuron

- **Structure :** The structure of artificial neural networks is inspired by biological neurons. A biological neuron has a cell body or soma to process the impulses, dendrites to receive them, and an axon that transfers them to other neurons. The input nodes of artificial neural networks receive input signals, the hidden layer nodes compute these input signals, and the output layer nodes compute the final output by processing the hidden layer's results using activation functions.

Biological Neuron	Artificial Neuron
Dendrite	Inputs
Cell nucleus or Soma	Nodes
Synapses	Weights
Axon	Output

- **Synapses** : Synapses are the links between biological neurons that enable the transmission of impulses from dendrites to the cell body. Synapses are the weights that join the one-layer nodes to the next-layer nodes in artificial neurons. The strength of the links is determined by the weight value.
- **Learning** : In biological neurons, learning happens in the cell body nucleus or soma, which has a nucleus that helps to process the impulses. An action potential is produced and travels through the axons if the impulses are powerful enough to reach the threshold. This becomes possible by synaptic plasticity, which represents the ability of synapses to become stronger or weaker over time in reaction to changes in their activity. In artificial neural networks, backpropagation is a technique used for learning, which adjusts the weights between nodes according to the error or differences between predicted and actual outcomes.

Biological Neuron	Artificial Neuron
Synaptic plasticity	Backpropagations

- **Activation** : In biological neurons, activation is the firing rate of the neuron which happens when the impulses are strong enough to reach the threshold. In artificial neural networks, A mathematical function known as an activation function maps the input to the output, and executes activations.

1.3 Structure of ANN

An Artificial Neural Network is made up of several layers, each performing computations on the input data to extract meaningful patterns. Let's break this down:

- **Perceptron – The Basic Unit:** The perceptron is the smallest building block of an ANN and can be viewed as a single artificial neuron.

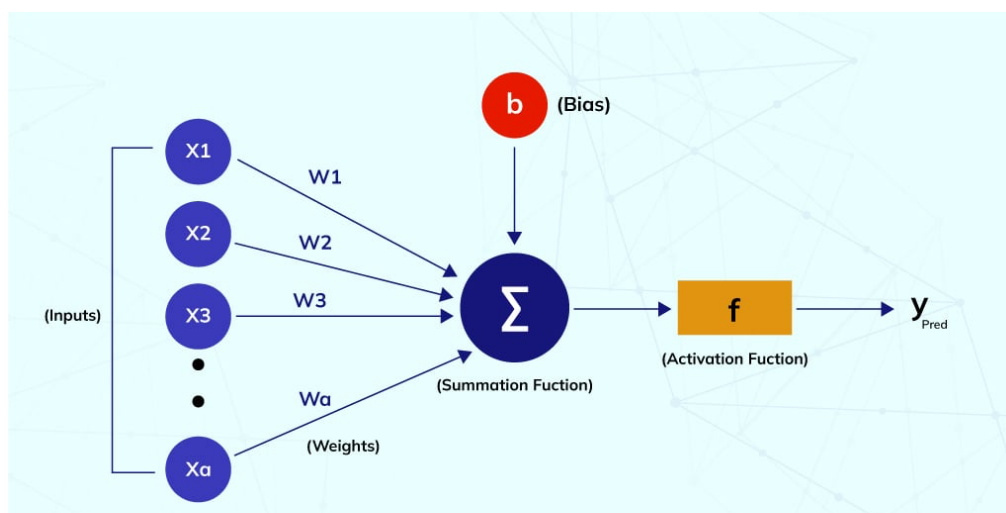


Figure 2 Perceptron

Here's how it works:

- **Inputs and Weights:** A perceptron receives multiple inputs ($x_1, x_2, x_3 \dots x_n$), (each multiplied by a corresponding weight ($w_1, w_2, w_3 \dots w_n$)) to signify the importance of each input.
- **Bias:** A bias term (b) is added to the weighted sum to adjust the output and increase model flexibility.
- **Activation Function:** The perceptron applies a function (like a step function) to the weighted sum to produce an output, typically 0 or 1. This output is passed to the next layer or used directly for classification.

Mathematical Representation:

$$\text{output} = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$$

where f is the activation function.

- **Limitation:** A single perceptron can only classify **linearly separable data** (i.e., data that can be separated by a straight line or hyperplane), which limits its use in complex problems. To address this, multi-layer architectures were introduced.

- **Layers of ANN:** An ANN typically consists of three main types of layers:

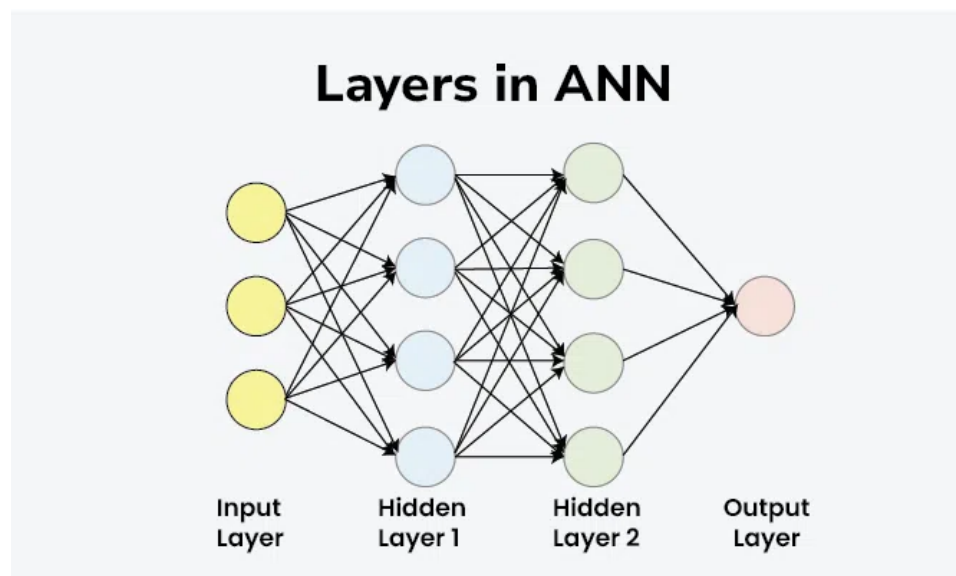


Figure 3 Layers in ANN

- **Input Layer:** The input layer is where the raw data (features of the problem) is introduced to the network. Each node in the input layer represents one feature.
 - **Hidden Layers:** These are intermediate layers between the input and output, where computations are performed. Multiple hidden layers can help the network learn more complex patterns, giving rise to what is called a **deep neural network (DNN)**.
 - **Output Layer:** The final layer of the network produces the desired output, whether it's a classification, regression result, or some other prediction.
- **Weights and Bias:** The network learns by adjusting **weights** assigned to each connection between neurons. The bias is another trainable parameter added to help shift the decision boundary.

1.4 Activation Functions

- **Purpose:** The activation function introduces **non-linearity** to the network, enabling it to learn complex patterns. Without activation functions, ANNs would behave like simple linear models.
- **Common Activation Functions:**
 - **Binary Step Function:** The binary step function outputs either 0 or 1, based on whether the input is below or above a certain threshold.

$$f(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$

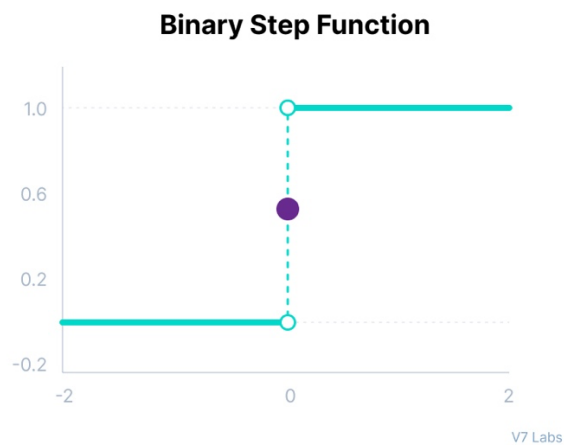


Figure 4 Binary step activation function

- **Characteristics:**
 - Commonly used in **binary classification** problems where a hard decision is required.
 - **Drawback:** Since it outputs a binary value (0 or 1), the binary step function has a **zero gradient** everywhere except at the threshold, making it unsuitable for learning with gradient-based methods like backpropagation.
- **Linear Activation Function:** The linear function outputs a value directly proportional to the input, without introducing any non-linearity.

$$f(x) = ax$$

where a is a constant.

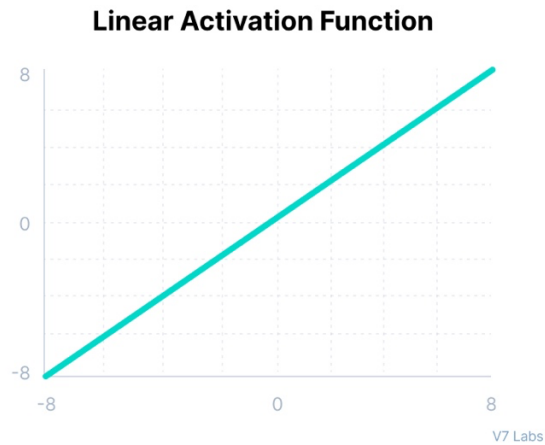


Figure 5 Linear activation function

- **Characteristics:**
 - Useful in the output layer for regression tasks, where we want a range of continuous values as output.
 - **Drawback:** Lacks non-linearity, meaning that if used in hidden layers, the entire network would act as a linear transformation, unable to capture complex patterns in data.
- **Sigmoid:** The sigmoid function maps any input to an output between 0 and 1, making it suitable for binary classification tasks.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Figure 6 Sigmoid activation function

- **Characteristics:**
 - Output is bounded between 0 and 1, making it useful when the network needs probabilities as output.

- **Drawback:** It can suffer from the **vanishing gradient problem**, where gradients become too small for deep networks to learn effectively during backpropagation.
- **ReLU (Rectified Linear Unit):** Efficiently handles positive values and helps mitigate the **vanishing gradient problem** in deep networks.

$$f(x) = \max(0, x)$$

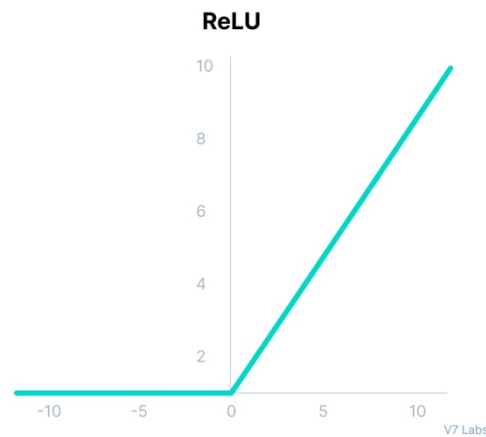


Figure 7 ReLu activation function

- **Characteristics:**
- Outputs zero for any negative input, allowing only positive values to pass through.
 - Helps mitigate the **vanishing gradient problem** by providing non-zero gradients for positive inputs, which keeps the network learning effectively.
 - **Drawback:** It can suffer from the **dying ReLU problem**, where neurons become inactive and only output zero.
- **Tanh (Hyperbolic Tangent):** Maps input values between -1 and 1, which helps centered data perform better.

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

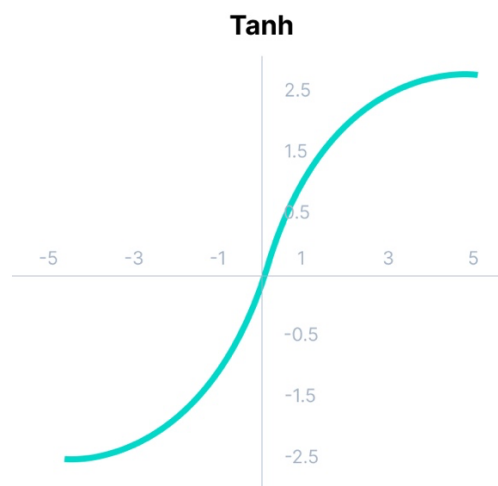


Figure 8 Tanh activation function

- **Characteristics:**

- Provides a symmetric output, which can help speed up convergence during training.
- **Drawback:** Like the sigmoid, tanh can suffer from the **vanishing gradient problem** in deep networks.

1.5 Training Neural Networks

To make the network learn from data, we need to adjust the weights and biases using a process called **training**.

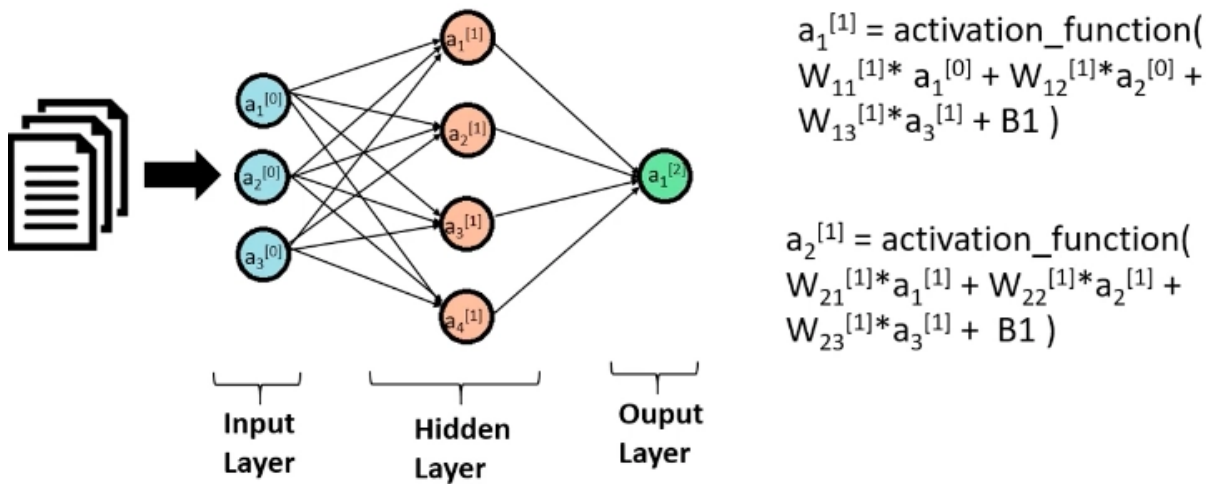


Figure 9 Training process of neural network

- **Forward Propagation:** During forward propagation, input data is passed through the network. Each neuron processes the inputs by applying weights, bias, and the activation function, passing the output to the next layer until the output layer is reached.
- **Loss Function:** The loss function measures how far the network's predictions are from the true values. For example:

- **Mean Squared Error (MSE)**

- **Use Case:** Regression tasks (where the output is continuous).
- **Description:** Measures the average squared difference between predicted and actual values. Squaring amplifies larger errors, making it sensitive to outliers.
- **Formula:**

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and N is the number of samples.

- Binary Cross-Entropy (Log Loss)

- **Use Case:** Binary classification tasks.
- **Description:** Measures the difference between two probability distributions for a binary classification problem.
- **Formula:**

$$\text{Binary Cross-Entropy} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Mean Absolute Error (MAE)

- **Use Case:** Regression tasks.
- **Description:** Measures the average absolute difference between predicted and actual values. Unlike MSE, it is less sensitive to outliers since it doesn't square the error.
- **Formula:**

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and N is the number of samples.

- **Backpropagation and Gradient Descent:**

- **Backpropagation:** Backpropagation is a core algorithm used in training artificial neural networks, enabling them to learn from errors and optimize weights in the network. It's essentially a method for efficiently calculating the **gradient of the loss function** with respect to the network's weights, allowing for gradient-based optimization techniques, like **Gradient Descent**.

1. Overview of Backpropagation Process

Backpropagation works in two main phases:

1. Forward Pass:

- The input data moves forward through the network, layer by layer, and the output is calculated.
- At each layer, the input is multiplied by weights, added with biases, and passed through an activation function.
- Once the network's final output is obtained, the **loss** (or error) between the predicted and actual output is computed using a **loss function**.

2. Backward Pass:

- The error (loss) from the forward pass is propagated backward through the network to update weights and biases.

- This backward propagation calculates the **gradient** of the loss function with respect to each weight by applying the **chain rule**.
- The calculated gradients indicate how each weight contributes to the error, which then guides adjustments to minimize the error.

2. Key Concepts in Backpropagation

a. Chain Rule of Calculus

- Backpropagation relies on the **chain rule**, a principle of calculus that allows us to find the derivative of a composition of functions.
- In neural networks, each layer's output is a function of its input, which is itself a function of the previous layer's output, and so on.
- The chain rule allows us to compute gradients layer-by-layer, starting from the output layer back to the input layer.

b. Partial Derivatives and Gradients

- A **partial derivative** measures the rate of change of a function with respect to one of its variables while keeping others constant.
- In backpropagation, we calculate the partial derivative of the loss with respect to each weight in the network.
- These derivatives, or **gradients**, help us understand how to adjust each weight to reduce the overall error.

c. Weight Updates

- After calculating gradients, we adjust the weights in the opposite direction of the gradient, aiming to minimize the error. This is done using a **learning rate** (α), which controls the size of the steps taken during weight updates.
- **Weight update formula:** $\omega_{new} = \omega_{old} - \alpha \cdot \frac{\partial L}{\partial \omega}$
where $\frac{\partial L}{\partial \omega}$ is the gradient of the loss with respect to weight ω .

d. Activation Functions and Their Derivatives

- Different activation functions (like Sigmoid, Tanh, ReLU) have different derivatives, which affect how gradients are calculated.
- The **derivative** of the activation function is used in backpropagation to update weights more precisely.

3. Backpropagation Steps in Detail

1. Calculate the Loss in the Output Layer:

- Compute the **loss** between the predicted output and the actual target using a suitable loss function (e.g., Mean Squared Error for regression or Cross-Entropy for classification).

2. Compute Gradients for the Output Layer:

- Calculate the gradient of the loss with respect to the weights in the output layer by using the chain rule.
- This gradient reflects how changes in the output weights will impact the loss.

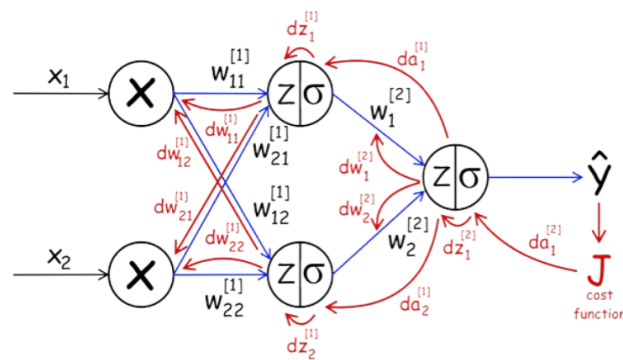
3. Backpropagate the Error to the Hidden Layers:

- Moving layer-by-layer backward, calculate the gradients for each preceding layer's weights.
- Each layer's gradient is influenced by both the activation function's derivative and the gradient of the previous layer.

4. Update the Weights Using Gradient Descent:

- After computing gradients for all layers, update each weight in the network using the calculated gradient and the learning rate.
- The learning rate determines the size of each update step: a smaller learning rate provides smaller steps and a more stable convergence, while a larger learning rate provides faster but potentially unstable steps.

4. Mathematical Example of Backpropagation



Backpropagation

Figure 10 Backpropagation

Consider a simple neural network with:

- **Input layer** with two neurons x_1 and x_2 ,
- **One hidden layer** with neurons h_1 and h_2
- **Output layer** with one neuron $\hat{y}$.

Forward Pass Calculations

1. **Hidden Layer Activations:**

$$h_1 = f(\omega_{1,1} \cdot x_1 + \omega_{1,2} \cdot x_2 + b_1)$$

$$h_2 = f(\omega_{2,1} \cdot x_1 + \omega_{2,2} \cdot x_2 + b_2)$$

2. **Output Layer Activation:**

$$\hat{y} = g(\omega_{o,1} \cdot h_1 + \omega_{o,2} \cdot h_2 + b_o)$$

3. **Loss Calculation** (e.g., Mean Squared Error if actual output is y):

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Backward Pass Calculations

1. **Gradient of Loss with Respect to Output Layer Weights:**

- Calculate $\frac{\partial L}{\partial \omega_{o,1}}$ and $\frac{\partial L}{\partial \omega_{o,2}}$ using the chain rule.

2. **Propagate Error to Hidden Layer:**

- Calculate gradients for $\omega_{1,1}, \omega_{1,2}, \omega_{2,1}, \omega_{2,2}$ by applying the chain rule and activation function derivatives.

3. **Update Weights:**

- Adjust all weights using the calculated gradients and learning rate.
- **Gradient Descent:** Gradient Descent is an optimization algorithm used to minimize the loss function in machine learning models. Gradient Descent adjusts the model's parameters by moving in the direction that reduces the loss. This is done by calculating the **gradient** (or slope) of the loss function with respect to each parameter and then updating those parameters in the opposite direction of the gradient.

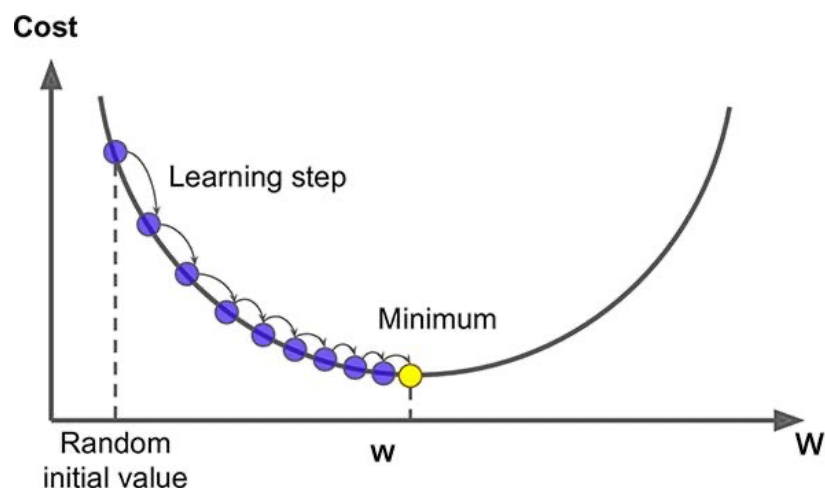


Figure 11 Gradient descent

1.6 Types of ANN Architectures

- **Single-Layer Perceptron (SLP):**

- Contains only one layer of neurons (the output layer) connected to the input. It can solve only linearly separable problems.
- **Multilayer Perceptron (MLP):**
 - This is the most common type of ANN. It contains one or more **hidden layers** between the input and output layers, allowing the network to learn complex, non-linear patterns.
 - An MLP with many hidden layers is referred to as a **deep neural network (DNN)**.
- **Recurrent Neural Networks (RNN):**
 - Used for sequential data, such as time series or text, where the output depends on previous inputs. The network has loops that allow information to persist.
- **Convolutional Neural Networks (CNN):**
 - Mainly used for image data. CNNs use **convolutional layers** to automatically detect important features like edges, shapes, and objects.

1.7 Limitations of ANN

- **Overfitting:** When a neural network is too complex (too many parameters) and learns not only the patterns in the training data but also the noise, leading to poor performance on new data. Regularization techniques (L2, L1, dropout) are used to prevent overfitting.
- **Computational Complexity:** Deep neural networks require large amounts of data and computational resources, which can be a challenge in real-world applications.
- **Long-Term Dependencies:** Traditional ANNs, especially basic architectures like SLPs or MLPs, struggle with tasks that require remembering long-term patterns, leading to the development of advanced models like **Recurrent Neural Networks (RNNs)**.

2. Recurrent Neural Networks (RNNs)

2.1 Introduction to RNNs

- **Recurrent Neural Network(RNN)** is a type of Neural Network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its **Hidden state**, which remembers some information about a sequence. The state is also referred to as *Memory State* since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

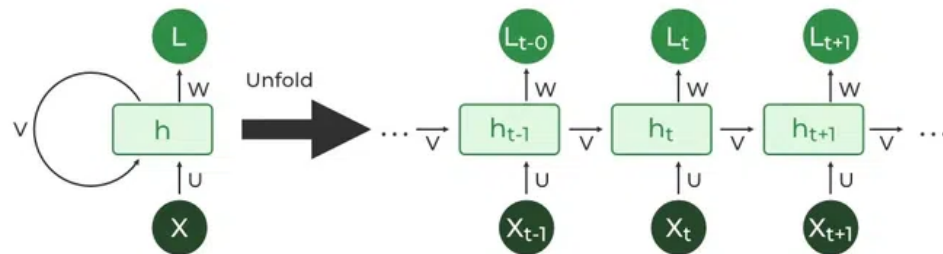


Figure 12 Recurrent neural network

- **How RNN differs from Feedforward Neural Network?**

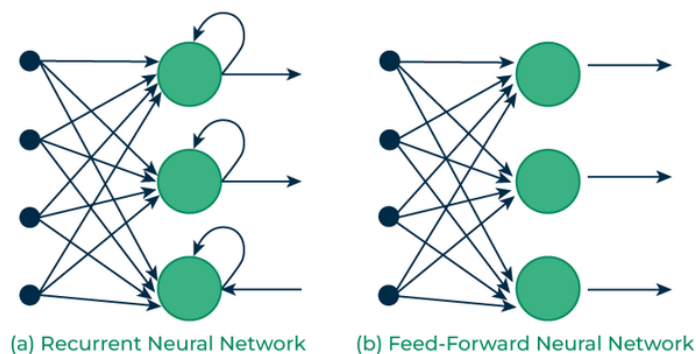


Figure 13 Recurrent vs Feedforward neural network

Artificial neural network that do not have looping nodes are called feed forward neural networks. Because all information is only passed forward, this kind of neural network is also referred to as a multi-layer neural network.

Information moves from the input layer to the output layer – if any hidden layers are present – unidirectionally in a feedforward neural network. These networks are appropriate for image

classification tasks, for example, where input and output are independent. Nevertheless, their inability to retain previous inputs automatically renders them less useful for sequential data analysis.

○ Why RNNs?

In many real-world problems, we need to understand the context to make accurate predictions.

For instance:

- In **natural language processing (NLP)**, understanding the meaning of a word depends on the words around it.
- In **time-series analysis** (like stock prices), future values may depend on previous data points.
- In **speech and audio processing**, each sound or phoneme is influenced by prior sounds.

RNNs are designed to handle these types of tasks by maintaining a memory of previous inputs, which is crucial for interpreting and predicting sequences.

2.2 How RNNs Work

In an RNN, each neuron processes the input along with the **hidden state** from the previous step, allowing the network to **carry information** across multiple time steps. This mechanism enables the network to maintain a form of memory throughout the sequence.

Mathematical Representation

At each time step t , the RNN receives an input x_t and produces an output y_t . It also maintains a **hidden state** h_t , which carries information from previous time steps.

The operations at each time step can be represented as:

1. Calculating the Hidden State:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

where:

- x_t : input at time step t ,
- h_{t-1} : hidden state from the previous time step $t - 1$,
- W_{xh} : weight matrix for the input-to-hidden connection,
- W_{hh} : weight matrix for the hidden-to-hidden connection,
- b_h : bias term,
- f : activation function, typically a **Tanh** or **ReLU**.

2. Producing the Output:

$$y_t = g(W_{hy}h_t + b_y)$$

where g could be a softmax function (for classification) or another appropriate activation. This recursive nature, where each hidden state depends on the previous hidden state, creates a memory effect, allowing RNNs to accumulate information over time.

Key Concepts

- **Hidden State:** The hidden state h_t is like a running summary of all previous inputs. It updates at each time step and is crucial for maintaining the sequence's context.
- **Backpropagation Through Time (BPTT):** In training RNNs, the gradients are propagated back through each time step to adjust the weights. This method, known as **Backpropagation Through Time (BPTT)**, is more complex than standard backpropagation because it considers the dependencies across time steps.

2.3 Types of RNNs

RNNs come in several forms, depending on the input-output structure:

1. **Vanilla RNN:** The simplest form of RNN, which processes sequences one time step at a time and is best suited for tasks with straightforward input-output relations.
2. **LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Units):** These are advanced RNN variants designed to address some of the limitations of vanilla RNNs, such as the vanishing gradient problem (discussed below).

2.4 Use Cases for RNNs

1. **Natural Language Processing (NLP):** Tasks like **language translation**, **text generation**, **sentiment analysis**, and **speech recognition** leverage RNNs to understand context within sequences of words or sounds.
2. **Time-Series Analysis:** RNNs are used for tasks like **stock price prediction**, **weather forecasting**, and **anomaly detection** in time-series data, where each data point is influenced by previous points.
3. **Video Processing:** RNNs can be applied to sequentially analyze frames in video data for tasks like **activity recognition** and **motion prediction**.